

ERP.1: Документи

Техноробочий проект

на розробку та впровадження підсистеми
електронного документообігу та обміну СЕВ
ОСС

Згідно з Постановою КМУ № 205 від 21.02.2025

Формалізація вимог за стандартом ISO/IEC/IEEE 29148:2018

Зміст

Зміст

1. Загальні відомості про засіб інформатизації

1.1. Найменування та підстави для розробки

Найменування засобу інформатизації: Підсистема «ERP.1: Документи» (далі — Модуль «Документи» або СЕД).

Відомості про замовника: ТОВ "Системи Електронного Урядування".

Підстави для виконання робіт:

- Закон України «Про електронний документообіг»;
- Постанова Кабінету Міністрів України від 17 січня 2018 року № 55 «Деякі питання документування управлінської діяльності»;
- Постанова Кабінету Міністрів України від 21 лютого 2025 року № 205 «Деякі питання створення, адміністрування та забезпечення функціонування засобу інформатизації».

1.2. Призначення та цілі створення засобу

Модуль «Документи» призначений для організації проходження та автоматизації життєвого циклу електронних документів, візування, підписання КЕП, контролю за виконанням завдань, а також міжвідомчого поштового обміну за стандартами X.420 / X.422.3.

Цілі створення засобу:

- Автоматизація вхідної, вихідної та внутрішньої кореспонденції;
- Контроль виконавської дисципліни за допомогою резолюцій керівництва;
- Підвищення прозорості та швидкості погодження проектів документів;
- Інтеграція з СЕВ OCC через систему електронної взаємодії;
- Надійне та масштабоване збереження документів у СУБД Mnesia та RocksDB.

1.3. План-графік виконання етапів робіт

1. **Етап 1: Розробка архітектурних рішень та інтерфейсів** — Створення карти продукту в N2O.DEV, дизайн-прототипів у Figma. (1.5 місяці).
2. **Етап 2: Реалізація FSM бізнес-процесів** — Написання Elixir BPE специфікацій та ABAC правил доступу. (2 місяці).
3. **Етап 3: Створення сховища даних та КЕП сервісів** — Опис рекордів Mnesia, RocksDB схем, інтеграція з бібліотеками валідації КЕП/КЕПч. (1.5 місяці).
4. **Етап 4: Інтеграція з СЕВ OCC** — Налаштування шлюзу обміну поштовими пакетами X.420. (2 місяці).
5. **Етап 5: Тестування та приймально-здавальні випробування** — Проведення Unit, UAT та тестів надійності. (1 місяць).

2. Відомості про робочий процес та умови експлуатації

2.1. Опис бізнес-процесів (BPMN / FSM)

2.1.1. Процес обробки вхідних документів (INPUT)

Опис кроків та станів (BPE FSM Transitions):

1. **Created (Чернетка):** Реєстратор створює картку вхідного документа. Перехід на наступний крок можливий лише після накладання системного КЕП/КЕПч та заповнення обов'язкових полів.
2. **Registration (Реєстрація):** Присвоєння вхідного реєстраційного номера за формулою журналу.
3. **gwNeedsDetermination (Умовний шлюз):** Автоматична перевірка: якщо документ терміновий (`urgent == true`), він перенаправляється на етап `UrgentConsideration`, інакше — на `InitialConsideration`.
4. **InitialConsideration (Попередній розгляд):** Реєстратор визначає виконавця або керівника для накладання резолюції.
5. **Resolution (Резолюція):** Керівник накладає доручення (задачу).
6. **Implementation (Виконання):** Створюється бізнес-задача `bizTask`.
7. **Completed (Виконано):** Документ списується в архів.

2.1.2. Процес обробки вихідних документів (OUTPUT)

Опис кроків та умов переходів:

1. **Draft (Чернетка):** Створення проекту документа виконавцем, додавання файлів (`attachments`).
2. **Coordination (Візування):** Процес внутрішнього погодження. Якщо хоча б один візанти відхиляє проект (`reject == true`), процес повертається на крок `Draft` з коментарем.
3. **Signing (Підписання):** Передача керівнику для накладання КЕП.
4. **Registration (Реєстрація):** Автоматична реєстрація вихідного номера.
5. **Dispatch (Відправка):** Передача через СЕВ OCC (конверт X.420).

2.1.3. Процес внутрішніх документів (INTERNAL)

1. **Draft:** Створення внутрішньої записки чи проекту наказу.
2. **Coordination:** Візування посадовими особами.
3. **Signing:** Підписання КЕП керівником.
4. **Registration:** Реєстрація в журналі внутрішніх документів.
5. **Distribution:** Автоматична розсилка адресатам усередині установи для ознайомлення чи виконання.

2.1.4. Організаційно-розпорядчі документи (ORG)

1. **Draft:** Створення проекту наказу/розпорядження.

2. **JuridicalApproval:** Обов'язковий юридичний аналіз юридичним департаментом.
3. **Coordination:** Погодження членами колегії чи заступниками.
4. **Signing:** Підписання КЕП головою установи.
5. **Registration:** Реєстрація та розсилання виконавцям із автоматичним створенням контрольних задач (ControlTask).

2.1.5. Контроль виконання доручень (CONTROLTASK)

1. **ControlStart:** Автоматичне взяття задачі на контроль.
2. **Monitoring:** Регулярний аналіз прогресу виконання контролюючим органом.
3. **Warning:** Автоматичне сповіщення виконавця за 3 дні до закінчення терміну виконання.
4. **Completed:** Зняття з контролю після погодження звіту контролером.

2.1.6. Виконання задач співвиконавцями (EXECTASK)

1. **TaskReceived:** Отримання завдання виконавцем.
2. **CoordinationWithChief:** Підготовка проміжних рішень співвиконавцями.
3. **ReportPreparation:** Написання звіту про виконання та прикріплення відповіді.
4. **Submission:** Передача результату головному виконавцю.

2.1.7. Розгляд звернень громадян (ADMISSION)

1. **AdmissionReceived:** Реєстрація звернення згідно з КОАТУУ та категоріями заявників.
2. **Consideration:** Попередній розгляд та резолюція (термін розгляду до 30 календарних днів).
3. **ResponseDrafting:** Підготовка відповіді заявнику.
4. **Dispatch:** Відправка відповіді КЕП та списання справи.

2.1.8. Додаткові бізнес-процеси (ADDITIONAL)

1. **Start:** Реєстрація додаткових запитів чи адвокатських запитів (термін виконання — 5 робочих днів).
2. **Execution:** Асинхронне отримання довідок від підрозділів.
3. **Response:** Відправка результату.

2.2. Опис ролей та прав доступу (ABAC)

Автентифікація та авторизація виконуються на базі ABAC атрибутів:

- **Правило 1 (Реєстратор):**

```
subject.role contains :registrar ∧ object.entity_type == :inputOrder
  ∧ object.status == :draft → Permit: create, read, write, register.
```

- **Правило 2 (Виконавець):**

```
subject.role contains :executor ∧ object.entity_type == :bizTask
  ∧ (object.executor == subject.id ∨ object.subexecutors contains subject.id)
  → Permit: read, write (виконання, прикріплення відповідей).
```

- **Правило 3 (Керівник):**

`subject.role contains :head ∧ object.entity_type == :inputOrder`

→ Permit: read, write (накладання резолюцій, підписання КЕП).

- **Правило 4 (Помічник/Асистент):**

`subject.role contains :assistant ∧ subject.substitute == object.head`

→ Permit: read, write (підготовка проектів резолюцій/підписів).

2.3. Умови експлуатації та технологічний стек

- Програмна платформа: Erlang/OTP версії 26 та Elixir версії 1.15.
- База даних: СУБД Mnesia для оперативних таблиць, RocksDB як сховище KVS для довгострокового зберігання історичних ланцюжків та бінарних об'єктів.
- Інтерфейс користувача: Web-клієнт на базі протоколів N2O та Nitro UI.

3. Інформаційне та програмне забезпечення

3.1. Специфікація ERP реквізитної інформації (Records)

Нижче наведено детальні структури даних з include-файлів бібліотеки schema для збереження документів та довідників СЕД.

3.1.1. Вхідний документ (inputOrder)

```
-record(inputOrder, { id= [] :: binary(),
  guid= [] :: list(),
  urgent= [] :: [] | boolean(),
  date= [] :: [] | calendar:datetime(),
  xml = [] :: [] | binary(),
  hash = [] :: binary() | list(),
  signature = [] :: binary() | list(),
  document_type= <<"Вхідний документ"/utf8>> :: term(),
  nomenclature= [] :: term(),
  category= [] :: [] | list(),
  corr= [] :: [] | list(),
  branch= [] :: [] | list(),
  head= [] :: [] | #'Person'{},
  signed= [] :: [] | list(),
  number_out= [] :: list(),
  date_out= [] :: calendar:datetime(),
  content= [] :: list(),
  note= [] :: list(),
  dueDate= [] :: [] | calendar:datetime(),
  dueDateTime = [] :: list(),
  to= [] :: [] | #'Person'{},
  main_sheets= [] :: [] | list(),
  add_sheets= [] :: [] | list(),
  device = [] :: [] | list(),
  registered_by= [] :: [] | #'Person'{},
  coordination= [] :: list(),
  topic= [] :: [] | binary(),
  attachments = [] :: list(term()),
  seq_id = [] :: list(),
  reject_comment = [] :: binary(),
  created_by = [] :: term(),
  created = [] :: term(),
  modified_by = [] :: term(),
  modified = [] :: term(),
  bizTask_initiator = [] :: term(),
  toAttention = #assistantMark{},
  project = #project{} :: #project{}
}).
```

3.1.2. Вихідний документ (outputOrder)

```
-record(outputOrder, { id= [] :: binary(),
  guid= [] :: list(),
  urgent= [] :: [] | boolean(),
  date= [] :: [] | calendar:datetime(),
  xml = [] :: [] | binary(),
  hash = [] :: binary() | list(),
  signature = [] :: binary() | list(),
  document_type = [] :: tuple(),
  nomenclature= [] :: term(),
  purpose= [] :: [] | list(),
```

```

purpose_sev=[] :: [] | list(),
corr=[] :: [] | list(),
deputy=[] :: [] | list(),
sign=[] :: [] | list(),
executor = [] :: [] | #'Person'{},
branch = [] :: list(),
head = [] :: [] | #'Person'{},
content= [] :: list(),
approvers = [] :: list(#'Person'{}),
signatory = [] :: list(#'Person'{}),
send_type = [] :: list(),
sent = #sendInfo{} :: #sendInfo{},
registered_by=[] :: [] | #'Person'{},
date_send=[] :: [] | calendar:datetime(),
note=[] :: list(),
attachments = [] :: list(term()),
project = #project{} :: #project{},
seq_id = [] :: list(),
created_by = [] :: term(),
created = [] :: term(),
modified_by = [] :: term(),
modified = [] :: term(),
template = [] :: term(),
assistantProcessed = #assistantMark{},
toAttention = #assistantMark{}
}).

```

3.1.3. Внутрішній документ (internalDoc)

```

-record(internalDoc, { id= [] :: binary(),
  guid= [] :: list(),
  urgent=[] :: [] | boolean(),
  date=[] :: [] | calendar:datetime(),
  xml = [] :: [] | binary(),
  hash = [] :: binary() | list(),
  signature = [] :: binary() | list(),
  document_type = [] :: tuple(),
  nomenclature=[] :: term(),
  executor=[] :: [] | #'Person'{},
  approvers = [] :: list(#'Person'{}),
  branch=[] :: [] | list(),
  head=[] :: [] | #'Person'{},
  content=[] :: term(),
  signatory=[] :: [] | list(#'Person'{}),
  target=[] :: [] | #'Person'{},
  note=[] :: list(),
  attachments = [] :: list(term()),
  registered_by=[] :: [] | #'Person'{},
  project = #project{} :: #project{},
  seq_id = [] :: list(),
  created_by = [] :: term(),
  created = [] :: term(),
  modified_by = [] :: term(),
  modified = [] :: term(),
  template = [] :: term(),
  bizTask_initiator = [] :: term(),
  toAttention = #assistantMark{}
}).

```

3.1.4. Бізнес-задача / Резолюція (bizTask)

```

-record(bizTask, { id= [] :: binary(),
  task_type = [] :: [] | list(),
  initiator = [] :: [] | #'Person'{},
  point=[] :: [] | list(),
  content = [] :: list(),
  document_type= <<"Задача"/utf8>> :: term(),
  nomenclature=[] :: term(),
  sign = [] :: [] | boolean(),

```



```

executor=[] :: [] | #'Person'{},
subexecutors = [] :: [] | list('Person'{}),
notify = [] :: [] | list('Person'{}),
fromDate = [] :: [] | calendar:datetime(),
dueDate = [] :: [] | calendar:datetime(),
dueDate_subexecutors = [] :: [] | calendar:datetime(),
priority = [] :: term(),
sent = #sendInfo{},
taskTime = [] :: integer(),
control=[] :: [] | boolean(),
period=[] :: [] | list(),
date_start=[] :: [] | calendar:datetime(),
date_finish=[] :: [] | calendar:datetime(),
date_control=[] :: [] | calendar:datetime(),
control_by=[] :: [] | list('Person'{}),
comment=[] :: [] | list(),
remove= false :: boolean(),
registered_by=[] :: [] | #'Person'{},
removed_by=[] :: [] | #'Person'{},
removed_date=[] :: [] | calendar:datetime(),
date_complete=[] :: [] | calendar:datetime(),
controller_comment=[] :: [] | list(),
complete_comment=[] :: [] | list(),
progress = [] :: [] | list(),
attachments = [] :: list(term()),
seq_id = [] :: list(),
parent = [] :: term(),
project = [] :: term(),
created_by = [] :: term(),
created = [] :: term(),
modified_by = [] :: term(),
modified = [] :: term(),
notifyTaskExist = false :: false | boolean(),
toAttention = #assistantMark{}
}).

```

3.1.5. Співробітник (Employee)

```

-record('Employee', { id
                    = kvs:seq([],[]) :: term(),
                    person
                    = [] :: [] | #'Person'{},
                    org
                    = [] :: [] | #'Organization'{},
                    branch
                    = [] :: #'Branch'{} | list(),
                    code
                    = [],
                    surname
                    = [],
                    name
                    = [],
                    middle_name
                    = [],
                    sex
                    = [],
                    birthday
                    = [],
                    hired
                    = [],
                    fired
                    = [],
                    inn
                    = [],
                    position
                    = [] :: list(),
                    number
                    = [],
                    group
                    = [] :: list(),
                    role
                    = [] :: list(atom()),
                    substitute
                    = [] :: [] | #'Person'{},
                    substitutes
                    = [] :: list(),
                    rightsDelegation
                    = [] :: list() | term(),
                    phone
                    = [] :: list() | binary(),
                    mail
                    = [] :: list(),
                    status
                    = [] :: term(),
                    type
                    = [] :: term() }).

```

3.1.6. Організація (Organization)

```

-record('Organization', { id = [] :: list(),
                        name
                        = [] :: [] | binary(),
                        details
                        = [] :: [] | binary(),
                        hq
                        = "00000" :: term(),

```

```

orgname    = [] :: [] | binary(),
address    = [] :: [] | binary(),
house      = [] :: [] | binary(),
director   = [] :: [] | term(),
position   = [] :: [] | binary(),
phones     = [] :: list() | binary(),
fax        = [] :: list() | binary(),
url        = [] :: [] | string(),
code       = [] :: list() | binary(),
login      = [] :: [] | binary(),
password   = [] :: [] | binary(),
location   = [] :: [] | #'Loc'{},
type       = [] :: term(),
keys       = 50 :: integer(),
email      = [] :: [] | binary(),
feed       = [] :: [] | binary(),
sendType   = [] :: [] | term() }.

```

3.1.7. Фізична особа (Person)

```

-record('Person', { id      = kvs:seq([],[]) :: [] | term(),
                   cn       = [] :: [] | binary(),
                   name     = [] :: [] | binary(),
                   displayName = [] :: [] | binary(),
                   location  = [] :: #'Loc'{},
                   hours    = 00 :: integer(),
                   type      = [] :: term() }).

```

3.1.8. Департамент / Філія (Branch)

```

-record('Branch', { id      = [] :: [] | binary(),
                   name     = [] :: list(),
                   org       = [] :: [] | binary(),
                   parent    = [] :: [] | binary(),
                   short     = [] :: list(),
                   head      = [] :: [] | #'Person'{},
                   deputies  = [] :: list(#'Person'{}),
                   type      = [] :: [] | atom(),
                   lvl       = 1 :: integer(),
                   index     = [] :: [] | binary(),
                   loc       = [] :: [] | #'Loc'{} }).

```

3.2. Програмне забезпечення (Реалізація)

3.2.1. newInputProc.ex (Процес обробки вхідних документів)

```

defmodule NewInput.Proc do
  import General.Proc
  import Doclink
  require BPE
  require ERP

  def def(), do: BPE.xml("/priv/bpmn/newInputProc.bpmn")
  def auth(_roles), do: true

  def action(BPE.broadcastEvent(), BPE.process() = proc) do
    BPE.result(type: :stop, state: proc)
  end

  def action(BPE.messageEvent(), BPE.process(userStarted: :system) = proc) do
    BPE.result(type: :stop, state: proc)
  end

  def action(BPE.messageEvent(payload: {:next, _}) = r, BPE.process() = proc) do

```

```

    NewOutput.Proc.action(r, proc)
end

def action(BPE.messageEvent(name: 'DocumentStatistic',
    payload: {executed, executors}), BPE.process(id: pid) = proc) do
    case :bpe.head(pid) do
        BPE.hist(task: BPE.sequenceFlow(source: s, target: t)) ->
            NewOutput.Proc.personal_statistic({:callback, s, t},
                BPE.result(executed: executed,
                    state: BPE.process(proc, executors: executors)), :document)
        _ -> []
    end
    BPE.result(type: :stop, state: proc)
end

def action(BPE.messageEvent(name: 'RemoveDocumentStatistic',
    payload: {stage, executors}), BPE.process() = proc) do
    NewOutput.Proc.remove_personal_statistic(:executed, :document,
        proc, stage, executors)
    BPE.result(type: :stop, state: proc)
end

def action(BPE.messageEvent(name: 'Implementation', payload: initiator),
    BPE.process(id: pid, docs: [doc | _]) = proc) do
    newDoc = :kvs.setfield(doc, :project,
        CRM.KEP.projAction(:execute, :kvs.field(doc, :project), initiator, []))
    BPE.result(state: BPE.process(proc, docs: [newDoc]),
        continue: [BPE.continue(fn: :next, args: [pid])])
end

def action(BPE.messageEvent(name: 'Executed',
    sender: BPE.process(id: pid, docs: [doc | _])),
    BPE.process(id: subj) = proc) do
    BPE.result(state: proc, continue: [
        BPE.continue(fn: :generateHistory, type: :spawn, module: :'Elixir.History',
            args: [%HistExt{action: :executed, procId: subj, subPid: pid,
                newValue: doc}]),
        BPE.continue(fn: :next, args: [subj])
    ])
end

def action(BPE.messageEvent(name: 'TaskCreated',
    payload: ERP.notifyTask(notify: n)), BPE.process(docs: [doc | _]) = proc) do
    newDoc = :kvs.setfield(doc, :modified_by, n ++
        case :kvs.field(doc, :modified_by) do
            x when is_list(x) -> x; _ -> []
        end)
    BPE.result(type: :stop, state: BPE.process(proc, docs: [newDoc]))
end

def action(BPE.messageEvent(), BPE.process() = proc) do
    BPE.result(type: :stop, state: proc)
end

def action(BPE.asyncEvent(name: n) = ev, BPE.process() = proc)
    when n in ['Reject', 'gwReject', 'Edit'] do
    NewOutput.Proc.action(ev, proc)
end

def action(BPE.asyncEvent(name: s) = r, state)
    when s in ['RemoveStatus', 'SetStatus'] do
    NewOutput.Proc.action(r, state)
end

def action(BPE.asyncEvent(name: 'ImplementationBlock',
    sender: BPE.process(id: pid),
    payload: ERP.bizTask(initiator: i, executor: e,
        subexecutors: s, notify: n, control_by: c)),
    BPE.process(id: subj, docs: [doc | _]) = proc) do
    :bpe.gw_block(pid, 'gwConfirmation', subj)
end

```

```

newDoc = :kvs.setfield(doc, :modified_by,
  :lists.flatten([e, s, n, c]) ++
  case :kvs.field(doc, :modified_by) do
    x when is_list(x) -> x; _ -> []
  end) |> :kvs.setfield(:bizTask_initiator, i)
BPE.result(type: :stop, state: BPE.process(proc, docs: [newDoc]))
end

def action(BPE.asyncEvent(name: 'ImplementationBlock',
  sender: BPE.process(id: pid)), BPE.process(id: subj) = proc) do
  :bpe.gw_block(pid, 'gwConfirmation', subj)
  BPE.result(type: :stop, state: proc)
end

def action(BPE.asyncEvent(name: 'Delete', payload: u),
  BPE.process(id: pid, parent: p, docs: [doc | _], name: n) = proc) do
  u != [] && History.generateHistory(%HistExt{action: :delete_proc,
    procId: p, user: u, docNum: RMK.Control.rowGetName(doc, n)})
  BPE.result(state: proc, continue: [
    BPE.continue(type: :stop),
    BPE.continue(type: :spawn, fn: :delete_process,
      module: "Elixir.General.Proc", args: [pid])
  ])
end

def action(BPE.asyncEvent(name: 'History') = ev, BPE.process() = proc) do
  NewOutput.Proc.action(ev, proc)
end

def action(BPE.asyncEvent(), proc) do
  BPE.result(type: :stop, state: proc)
end

def action({:request, _, _} = r,
  BPE.process(userStarted: :system) = proc) do
  BPE.result(type: :stop, state: generalAction(r, proc, proc))
end

def action({:request, 'Created', 'Registration'} = r,
  BPE.process(id: pid, docs: [doc | _], parent: p) = proc0) do
  {_, proc} = :bpe.ensure_mon(proc0)
  ERP.project(comment: newComment) = rejectComment(:kvs.field(doc, :project))
  newDoc = :kvs.setfield(doc, :project, ERP.project(comment: newComment))
  :bpe.broadcastEvent(pid, BPE.broadcastEvent(name: 'Delete', sender: proc))
  saveChildProc(pid, p)
  CRM.Search.Common.save_index(doc, [], pid)
  CRM.Search.Common.save_index([:corr, :number_out, :date_out], doc, [], pid)
  newProc = BPE.process(proc, docs: [newDoc])
  :bpe.load(p) != [] && :bpe.messageEvent(p,
    BPE.messageEvent(name: 'Registration', payload: newProc))
  BPE.result(state: generalAction(r, proc, newProc),
    continue: [
      continueGeneral(r, newProc, newDoc,
        if newComment == [] do :created else [] end),
      BPE.continue(type: :stop)
    ])
end

def action({:request, 'Registration', 'gwNeedsDetermination'} = r,
  BPE.process(id: pid, executors: e, docs: [doc | _]) = proc) do
  newDoc = :kvs.setfield(doc, :project, rejectComment(:kvs.field(doc, :project)))
  newProc = BPE.process(proc, docs: [newDoc])
  indexUrgent(newProc, newDoc)
  BPE.result(state: generalAction(r, proc, newProc), executed: e,
    continue: [
      continueGeneral(r, newProc, newDoc),
      BPE.continue(fn: :next, args: [pid])
    ])
end

```

```

def action({:request, 'InitialConsideration', _} = r,
  BPE.process(executors: e, docs: [doc | _]) = proc) do
  newDoc = :kvs.setfield(doc, :project, rejectComment(:kvs.field(doc, :project)))
  newProc = BPE.process(proc, docs: [newDoc])
  BPE.result(state: generalAction(r, proc, newProc), executed: e,
    continue: [
      continueGeneral(r, newProc, newDoc),
      BPE.continue(type: :stop)
    ])
end
end
end

```

3.2.2. newOutputProc.ex (Процес обробки вихідних документів)

```

defmodule NewOutput.Proc do
  require FORM
  require BPE
  require ERP
  require CRM.KEP
  require KVS
  require General.Proc
  require Reports
  import General.Proc

  @sev_send_type '170'

  def def(), do: BPE.xml("/priv/bpmn/newOutputProc.bpmn")
  def auth(_roles), do: true

  def general_specific_statistic({:callback, _, _}, result, _) do: result
  def personal_specific_statistic({:callback, _, _}, result, _) do: result
  def general_statistic({:callback, _, _}, result, _) do: result
  def main_document_statistic({:callback, _, _}, result, _) do: result
  def personal_statistic({:callback, _, _}, result, _) do: result
  def remove_personal_statistic(_, _, _, _, _), do: []

  def history({:callback, _, t}, BPE.result(executed: executed,
    state: BPE.process(id: pid, parent: parent, docs: [doc | _],
      executors: executors)) = result, :created) do
    History.generateHistory(%HistExt{action: :created, procId: pid,
      user: executed, userTo: executors, stage: t, newValue: doc})
    History.generateHistory(%HistExt{action: :add_proc, procId: parent,
      subPid: pid, user: :kvs.field(doc, :registered_by)})
    result
  end

  def history({:callback, _, _}, BPE.result(state: BPE.process(id: pid,
    docs: [doc | _], parent: p)) = result, :familiarization) do
    History.generateHistory(%HistExt{action: :familiarization, procId: pid,
      newValue: doc})
    History.generateHistory(%HistExt{action: :familiarized, procId: p,
      newValue: doc, subPid: pid})
    result
  end

  def history({:callback, _, _}, BPE.result(executed: executed,
    state: BPE.process(id: pid, docs: [doc | _])) = result, action)
    when action in [:agreed, :signed, :certified, :execution] do
    History.generateHistory(%HistExt{action: action, procId: pid,
      user: executed, newValue: doc})
    result
  end

  def history({:callback, _, t}, BPE.result(executed: executed,
    state: BPE.process(id: pid, executors: executors)) = result, _) do
    History.generateHistory(%HistExt{action: :stage, procId: pid,
      user: executed, userTo: executors, stage: t})
    result
  end
end

```

```

end

def report({:callback, _, _}, BPE.result(state: proc) = result, {action, t}) do
  :erlang.apply(Reports, action, [Reports.info(report_type: t, process: proc)])
  result
end

def action(BPE.broadcastEvent(name: 'ImplementationRejected'), proc) do
  BPE.result(state: proc, continue: [
    BPE.continue(fn: :add, type: :spawn, module: :Elixir.Reports',
      args: [Reports.info(report_type: :expired, types: :implementation,
        process: proc)])
  ])
end

def action(BPE.broadcastEvent(name: 'Delete',
  sender: BPE.process(module: m)) = ev, BPE.process(id: pid) = proc)
  when m in [NewInput.Proc, NewOrg.Proc, NewInternal.Proc] do
  :bpe.broadcastEvent(pid, ev)
  BPE.result(state: proc, continue: [
    BPE.continue(type: :stop),
    BPE.continue(type: :spawn, fn: :delete_process,
      module: :Elixir.General.Proc", args: [pid])
  ])
end

def action(BPE.broadcastEvent(name: 'Delete') = ev,
  BPE.process(id: pid) = proc) do
  :bpe.broadcastEvent(pid, ev)
  BPE.result(type: :stop, state: proc)
end

def action(BPE.asyncEvent(name: 'SetStatus', payload: {group, s, user}),
  BPE.process(id: pid, module: m, executors: execs, docs: [doc | _]) = proc) do
  request = case :bpe.head(pid) do
    BPE.hist(task: BPE.sequenceFlow(source: src, target: dst)) ->
      {:request, src, dst}
    _ -> []
  end
  newProc = BPE.process(proc, executors: setExecutorStatus(execs, user, s))
  m.generalMoveTo(request, newProc, doc, "#{group}")
  BPE.result(state: newProc, continue: [
    continueGeneral(request, newProc, doc, group),
    BPE.continue(fn: :generateHistory, type: :spawn, module: :Elixir.History',
      args: [%HistExt{action: s, procId: pid, user: user}])
  ])
end
end

```

3.2.3. abacPDP.ex (Модуль прийняття рішень доступу)

```

defmodule ABAC.PDP do
  require ABAC
  require KVS

  def get_policy([ABAC.policy() = h | t]), do: get_policy(h) ++ get_policy(t)
  def get_policy(ABAC.policy(object: o, api_endpoint: e)) do
    :lists.filter(fn ABAC.policy(api_endpoint: ^e) -> true; _ -> false end,
      :kvs.index(:policy, :object, o, KVS.kvs(mod: :kvs_mnesia)))
  end
  def get_policy(_), do: []

  def decision(req0) do
    {r, p} = ABAC.PIP.request(req0)
    :lists.any(fn x -> :lists.any(&policy(r, &1), get_policy(x)) end, p)
  end

  def deny(req0) do
    {r, p} = ABAC.PIP.request(req0)
    :lists.any(fn x -> :lists.any(&!policy(r, &1), get_policy(x)) end, p)
  end
end

```

```

end

def get(req0) do
  {ABAC.request() = req, policy} = ABAC.PIP.request(req0)
  :lists.flatten(:lists.map(fn x ->
    :lists.filtermap(fn
      ABAC.rule(values: []) -> false
      ABAC.rule(values: v, object: o) -> {true, :kvs.field(o, v)}
    end, policy(req, x))
  end, get_policy(policy)))
end

def parse(r), do: ABAC.PIP.parse(r)

def policy(ABAC.request(type: t) = req, ABAC.policy(rules: r, combining: c))
  when t in [:decision, :deny] do
    :erlang.apply(:lists, c, [
      fn ref -> rule(req, :erlang.element(2,
        :kvs.get(:rule, ABAC.rule_ref(ref, :id), KVS.kvs(mod: :kvs_mnesia)))) end,
      r
    ])
  end
def policy(ABAC.request(type: :get) = req, ABAC.policy(rules: r,
  combining: :all)) do
  :erlang.element(2, :lists.foldr(fn
    _, {false, _} = x -> x
    ABAC.rule_ref(id: i), {true, acc} ->
      rule = :erlang.element(2, :kvs.get(:rule, i, KVS.kvs(mod: :kvs_mnesia)))
      if rule(req, rule), do: {true, [rule | acc]}, else: {false, []}
    end, {true, []}, r))
end
def policy(ABAC.request(type: :get) = req, ABAC.policy(rules: r)) do
  :lists.foldr(fn ABAC.rule_ref(id: i), acc ->
    x = :erlang.element(2, :kvs.get(:rule, i, KVS.kvs(mod: :kvs_mnesia)))
    case rule(req, x) do
      true -> [x | acc]
      _ -> acc
    end
  end, [], r)
end
def policy(ABAC.request(type: :decision), _), do: false
def policy(ABAC.request(type: :deny), _), do: true
def policy(ABAC.request(type: :get), _), do: []

def rule(ABAC.request(endpoint: e, resources: resources) = req,
  ABAC.rule(api_endpoint: e, type: t, object_condition: [], object: o,
  subject_condition: c, resource_match: rm) = r) do
  x = :lists.filter(fn ~o -> true; _ -> false end, resources)
  result(t, x != [] and :erlang.apply(:lists, rm, [
    fn _ -> :erlang.apply("Elixir.ABAC.Condition", c, [req, r]) end,
    x
  ]))
end
def rule(ABAC.request(endpoint: e, resources: resources) = req,
  ABAC.rule(api_endpoint: e, type: t, object_condition: oc, object: o,
  subject_condition: c, resource_match: rm) = r) do
  x = :lists.filter(&(:erlang.is_record(&1, :erlang.element(1, o))), resources)
  result(t, x != [] and :erlang.apply(:lists, rm, [
    fn item -> :erlang.apply("Elixir.ABAC.Condition", c, [req, r]) and
      :erlang.apply("Elixir.ABAC.Condition", oc, [item, r]) end,
    x
  ]))
end
def rule(_, _), do: false

defp result(:permit, x), do: x
defp result(:deny, x), do: !x
end

```

3.2.4. `kep_validator.erl` (Модуль валідації електронних підписів)

```
-module(kep_validator).
-export([validate_signature/2, verify_time_stamp/1]).

validate_signature(FileHash, SignatureBytes) ->
  case ca_client:verify_signature(FileHash, SignatureBytes) of
    {ok, SignerCert} ->
      case ca_client:check_cert_status(SignerCert) of
        valid -> ok;
        expired -> {error, cert_expired};
        revoked -> {error, cert_revoked}
      end;
    Error -> Error
  end.

verify_time_stamp(TimeStampToken) ->
  case ca_client:verify_tsp(TimeStampToken) of
    true -> ok;
    false -> {error, invalid_tsp}
  end.
```

3.3. Інсталяція та конфігурація системи (`mix.exs`)

Запуск та збирання релізу системи електронного документообігу на базі Erlang/Elixir здійснюється наступним чином:

3.3.1. Пререквізити для операційної системи (Ubuntu)

```
$ sudo apt install erlang elixir build-essential libcsv3 libcsv-dev cmake unoconv
$ sudo snap install onlyoffice-ds
```

3.3.2. Залежності проекту (`mix.exs`)

```
defmodule CRM.Mixfile do
  use Mix.Project
  def project do
    [ app: :crm, version: "3.1.0", elixir: "> 1.9", deps: deps() ]
  end
  def deps do
    [
      {:cowboy, "~> 2.9.0"},
      {:cowlib, "~> 2.11.0"},
      {:rocksdb, "~> 1.6.0"},
      {:sax, "~> 1.0.0"},
      {:ca, "~> 1.11.0"},
      {:barlix2, "~> 0.6.0"},
      {:qrcode, "~> 0.1"},
      {:n2o, "~> 8.12.1", override: true},
      {:soa, "~> 0.1.7"},
      {:jsone, "~> 1.5.1"},
      {:bpe, "7.6.3"},
      {:kvs, "~> 9.7.0", override: true},
      {:smtp, "~> 1.1.2"},
      {:pop3, "~> 2.1.0"},
      {:nitro, "~> 7.7.1", override: true},
      {:form, "~> 7.7.0"}
    ]
  end
end
```


3.3.3. Компіляція та запуск у режимі демона

```
$ mix deps.get  
$ mix release  
$ _build/dev/rel/crm/bin/crm daemon
```

4. Вимоги до засобу за ISO/IEC/IEEE 29148

4.1. Функціональні вимоги

- **[REQ-MAIL-FUN-001]** Модуль «Документи» повинен автоматично нумерувати вхідні, вихідні та внутрішні документи відповідно до заданих шаблонів нумерації та календарного року.
- **[REQ-MAIL-FUN-002]** Модуль «Документи» повинен наносити QR-код (21x21 мм) та лінійний штрих-код на першу сторінку документа під час реєстрації.
- **[REQ-MAIL-FUN-003]** Модуль «Документи» повинен виконувати перевірку кваліфікованих електронних підписів (КЕП) та печаток (КЕПч) за протоколами OCSP/CRL.
- **[REQ-MAIL-FUN-004]** Модуль «Документи» повинен підтримувати паралельне виконання завдань (підзадач) співвиконавцями в межах однієї РМК.
- **[REQ-MAIL-FUN-005]** Модуль «Документи» повинен здійснювати автоматичний контроль термінів виконання та надсилати сповіщення при наближенні дати прострочення.
- **[REQ-MAIL-FUN-006]** Модуль «Документи» повинен забезпечувати інтеграцію з СЕВ OCC для відправки та прийому документів через конверти MHS X.420.
- **[REQ-MAIL-FUN-007]** Модуль «Документи» повинен дозволяти делегування прав керівника помічнику або заміснику із фіксацією періоду делегування.

4.2. Вимоги до інтерфейсів та дизайну

- **[REQ-MAIL-UI-001]** Користувацький інтерфейс повинен відповідати вимогам WCAG 2.1 рівню AA щодо вебдоступності.
- **[REQ-MAIL-UI-002]** Інтерфейс має бути побудований за технологією Single Page Application (SPA) на базі WebSocket з'єднань.

4.3. Вимоги до безпеки та захисту інформації

- **[REQ-MAIL-SEC-001]** Авторизація користувачів повинна відбуватися виключно за допомогою сертифікатів КЕП X.509.
- **[REQ-MAIL-SEC-002]** Доступ до будь-якої РМК повинен перевірятися підсистемою ABAC на основі атрибутів користувача та об'єкта.
- **[REQ-MAIL-SEC-003]** Вся взаємодія між клієнтом та сервером повинна здійснюватися через шифроване з'єднання TLS 1.3.

4.4. Вимоги до продуктивності

- **[REQ-MAIL-PERF-001]** Час відкриття та відображення сторінки РМК не повинен перевищувати 500 мс.
- **[REQ-MAIL-PERF-002]** Система повинна підтримувати обробку не менше 500 паралельних WebSocket з'єднань на один процесорний вузол.

4.5. Вимоги до логування, тестування та супроводу

- **[REQ-MAIL-LOG-001]** Усі дії користувачів над РМК (створення, редагування, підписання) повинні логуватися в системному Audit Log у форматі JSON.
- **[REQ-MAIL-ADM-001]** Покриття вихідного коду автоматичними тестами має бути не менше 80%.